

UNIT I	INTRODUCTION TO REST API
---------------	---------------------------------

RESTful APIs – overview about data exchange between client and server - separating concerns between handling HTTP requests and executing business logic - retrieving server resources via HTTP requests - injection of property values - self-contained application - serialization and deserialization – JSON properties - managing data access.
--

1. RESTful APIs:

RESTful APIs (Representational State Transfer) are a set of web service principles that allow communication between client and server over HTTP in a stateless, standardized manner.

Key Concepts:

<u>Term</u>	<u>Description</u>
Client	Makes requests (e.g., a browser, mobile app)
Server	Processes requests and returns responses (e.g., APIs, databases)
Stateless	Each request from client to server must contain all the information needed
HTTP Methods	GET, POST, PUT, DELETE – used to perform actions on resources
Resources	Data objects like users, products, etc., identified by URLs
Response Format	Typically, JSON or XML

HTTP Methods in REST:

Method	Description	Example URL
GET	Retrieve data	/api/products
POST	Create new data	/api/products
PUT	Update existing data	/api/products/123
DELETE	Delete existing data	/api/products/123

Real-Time Example: Online Shopping App

Let's consider Amazon-like e-commerce:

Example: Products API

1. GET all products

- **Request:** GET /api/products
- **Response:**

```
json
[
  { "id": 1, "name": "Laptop", "price": 599.99 },
  { "id": 2, "name": "Headphones", "price": 99.99 }
]
```

2. GET a single product

- **Request:** GET /api/products/1
- **Response:**

```
json
{ "id": 1, "name": "Laptop", "price": 599.99 }
```

3. POST (Add a new product)

- **Request:** POST /api/products

```
json
{ "name": "Mouse", "price": 25.00 }
```

- **Response:**

```
json
{ "id": 3, "name": "Mouse", "price": 25.00 }
```

4. PUT (Update product)

- **Request:** PUT /api/products/3

```
json
{ "name": "Wireless Mouse", "price": 30.00 }
```

5. DELETE

- **Request:** DELETE /api/products/3
- **Response:** 204 No Content

Summary

RESTful APIs:

- Use standard HTTP methods
- Are stateless and scalable
- Return data typically in JSON format
- Help power **web apps, mobile apps, IoT devices**, and more

2. Overview about data exchange between client and server

What is Happening?

In a **RESTful API**, the **client** sends an **HTTP request** to the **server**, and the **server** sends back an **HTTP response**, typically in **JSON** or **XML** format.

Data Exchange Components

Component	Description
Client	Initiates the request (browser, mobile app, Postman, frontend code, etc.)
Server/API	Handles the request, processes data, and sends a response
HTTP Request	Includes method (GET, POST, etc.), URL, headers, and body
HTTP Response	Includes status code, headers, and data (JSON/XML)

Real-Time Example: Student Information System

Goal: Retrieve student info

Client sends HTTP GET request:

GET /api/students/101 HTTP/1.1

Host: college.edu

Accept: application/json

Server receives the request and responds:

http

HTTP/1.1 200 OK

Content-Type: application/json

```
{  
  "id": 101,  
  "name": "Anjali",  
  "department": "CSE",  
  "year": "III"  
}
```

Common Exchange Patterns

Action	HTTP Method	Endpoint	Request Body	Response Example
Get a student	GET	/api/students/101	None	{ "id": 101, "name": "Anjali" }
Add a student	POST	/api/students	{ "name": ..., "dept": ... }	{ "id": 102, ... }
Update info	PUT	/api/students/101	{ "year": "IV" }	{ "id": 101, "year": "IV" }
Delete student	DELETE	/api/students/101	None	204 No Content

Summary

- **Client:** Sends HTTP request to REST endpoint.
 - **Server:** Receives, processes, and sends a structured response.
 - **Format:** Usually JSON for both request and response.
 - **Stateless:** Every request is independent; no session info stored between them.
-

3. Separating concerns between handling HTTP requests and executing business logic

What is Separation of Concerns?

Separation of Concerns means dividing code into distinct sections based on responsibility:

Concern	Responsibility
Controller (HTTP layer)	Handles HTTP requests/responses, routing, input validation
Service (Business layer)	Contains core logic/rules, calculations, decisions
Repository/DAO (Data layer)	Manages database operations (CRUD)

Why Separate?

- **Maintainability:** Easier to read and update
 - **Testability:** Logic can be tested without HTTP
 - **Reusability:** Business logic can be reused across APIs
 - **Scalability:** Layers can grow independently
-

Real-Time Example: Student Management System (Spring Boot)

Project Structure:

```
com.example.studentapi
|
|— controller
|   └── StudentController.java
|
|— service
|   └── StudentService.java
|
|— model
|   └── Student.java
|
|— repository
|   └── StudentRepository.java
|
└── StudentApiApplication.java
```

1. Controller – Handles HTTP

```
@RestController
@RequestMapping("/api/students")
public class StudentController {
    @Autowired
    private StudentService studentService;

    @GetMapping("/{id}")
    public ResponseEntity<Student> getStudent(@PathVariable Long id) {
        Student student = studentService.getStudentById(id);
        return ResponseEntity.ok(student);
    }
}
```

```
}
```

2. Service – Business Logic

@Service

```
public class StudentService {
```

```
    @Autowired
```

```
    private StudentRepository studentRepository;
```

```
    public Student getStudentById(Long id) {
```

```
        return studentRepository.findById(id)
```

```
            .orElseThrow(() -> new RuntimeException("Student not found"));
```

```
    }
```

```
}
```

3. Repository – Data Access

@Repository

```
public interface StudentRepository extends JpaRepository<Student, Long> {
```

```
}
```

4. Model – Student Class

@Entity

```
public class Student {
```

```
    @Id
```

```
    private Long id;
```

```
    private String name;
```

```
    private String department;
```

```
}
```

Summary Table

Layer	File/Class	Responsibility
Controller	StudentController	Handle request & send response
Service	StudentService	Contains business rules/logic
Repository	StudentRepository	Connects with DB (CRUD)
Model	Student	Entity representing data structure

4. Retrieving server resources via HTTP requests.

In the context of **RESTful APIs**, retrieving server resources means that a **client (like a browser or mobile app)** requests specific **data from the server** using an **HTTP GET request**.

Key Concepts

Term	Meaning
Client	The requester (e.g., browser, frontend app)
Server	Hosts resources and responds to client requests
Resource	Any piece of data (user, product, student, etc.) identified by a URL
HTTP GET	Method used to retrieve (read-only) resources
URI/Endpoint	The address that points to the resource (e.g., /api/products/1)
Response	Data returned, usually in JSON or XML format

Real-Time Example: Retrieving Student Info

Client Sends HTTP GET Request:

GET /api/students/101 HTTP/1.1

Host: college.edu

Server Responds with JSON:

HTTP/1.1 200 OK

Content-Type: application/json

```
{  
  "id": 101,  
  "name": "Anjali",  
  "department": "CSE"  
}
```

Typical Use Cases of Retrieving Resources

Resource	Endpoint Example	Description
All products	GET /api/products	List all products
Single user	GET /api/users/5	Get user details with ID 5
Order history	GET /api/orders?userId=10	Get orders of user 10
Search books	GET /api/books?title=java	Get books with "java" in the title

How It Works (Flow)

1. **Client** sends GET request to a REST API.
 2. **Server** looks up the resource (often from a database).
 3. **Server** responds with the data (usually JSON).
 4. **Client** receives and renders or processes the data.
-

Summary

- Retrieving server resources via HTTP is usually done with the GET method.
- The resource is specified by a **URI**.
- Response includes **status code** (e.g., 200 OK) and **data** (JSON/XML).
- This is fundamental to how web and mobile apps fetch and display data.

5. Injection of property values

In Spring Boot, **injection of property values** is the process of accessing values from the `application.properties` or `application.yml` configuration files in your code—typically into components like services, controllers, or configuration classes.

Why Property Injection?

It allows you to externalize configuration, making the application flexible and easier to manage for different environments (dev, test, prod).

Example: Injecting Property Values in a Spring Boot REST API

1. Add properties to `application.properties`

`properties`

```
app.name=MySpringApp
```

```
app.version=1.0.0
```

2. Create a Configuration Class to Hold These Properties

You can use `@Value` or `@ConfigurationProperties`. Here's how both work:

Option A: Using @Value

```
import org.springframework.beans.factory.annotation.Value;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RestController;
```

```
@RestController
```

```
public class AppInfoController {
```

```
    @Value("${app.name}")
```

```
    private String appName;
```

```
    @Value("${app.version}")
```

```
    private String appVersion;
```

```
    @GetMapping("/info")
```

```
    public String getAppInfo() {
```

```
        return "App Name: " + appName + ", Version: " + appVersion;
```

```
    }
```

```
}
```

Option B: Using @ConfigurationProperties

Step 1: Create a POJO for the properties

```
import org.springframework.boot.context.properties.ConfigurationProperties;
import org.springframework.stereotype.Component;
```

```
@Component
```

```
@ConfigurationProperties(prefix = "app")
```

```
public class AppProperties {
```

```
    private String name;
```

```
    private String version;
```

```
    // Getters and Setters
```

```
    public String getName() { return name; }
```

```
    public void setName(String name) { this.name = name; }
```

```
    public String getVersion() { return version; }
```

```
    public void setVersion(String version) { this.version = version; }
```

```
}
```

Step 2: Inject this class into your controller

```
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RestController;

@RestController

public class AppInfoController {

    private final AppProperties appProperties;

    public AppInfoController(AppProperties appProperties) {
        this.appProperties = appProperties;
    }

    @GetMapping("/info")
    public String getAppInfo() {
        return "App Name: " + appProperties.getName() + ", Version: " + appProperties.getVersion();
    }
}
```

Which Approach to Use?

Approach	Use When...
@Value	You have a few simple properties to inject.
@ConfigurationProperties	You want to bind a group of related properties. It supports nested properties and validation.

6. Self-contained application

What is a Self-Contained Application?

A **self-contained Spring Boot application** is one that can:

- **Run independently** (no external server like Tomcat needed)
- **Package everything** (server, configuration, dependencies, code) into a **single executable JAR**
- Be deployed and run with just: `java -jar myapp.jar`

Key Features of Self-Contained Spring Boot Apps:

Feature	Description
Embedded Web Server	Uses built-in Tomcat/Jetty/Undertow
Single JAR Packaging	All classes, libraries, and resources packed together
Auto Configuration	Spring Boot configures many components automatically
No Need for WAR or External Server	Eliminates traditional deployment to servlet containers
Production Ready	Actuator, logging, health checks can be built-in

How to Create a Self-Contained Spring Boot App

1. Generate a project via [Spring Initializr](#)

- Dependencies: Spring Web

2. Example Code

DemoApplication.java

```
package com.example.demo;
```

```
import org.springframework.boot.SpringApplication;
```

```
import org.springframework.boot.autoconfigure.SpringBootApplication;
```

```
@SpringBootApplication
```

```
public class DemoApplication {
```

```
    public static void main(String[] args) {
```

```
        SpringApplication.run(DemoApplication.class, args);
```

```
    } }
```

HelloController.java

```
package com.example.demo;

import org.springframework.web.bind.annotation.*;

@RestController

public class HelloController {

    @GetMapping("/hello")
    public String hello() {
        return "Hello from a self-contained Spring Boot app!";
    }
}
```

Build the JAR File

./mvnw clean package

You'll find the JAR in:

target/demo-0.0.1-SNAPSHOT.jar

Run It

java -jar target/demo-0.0.1-SNAPSHOT.jar

Then visit:

<http://localhost:8080/hello>

Output

Hello from a self-contained Spring Boot app!

Summary

Benefit

Self-Contained App

Embedded Server	•
Standalone Execution	• (via java -jar)
Easy Deployment	• (just 1 JAR)
Minimal Setup	• (thanks to Spring Boot auto-config)

7. Serialization and deserialization

What Is Serialization?

Serialization is the process of **converting a Java object to a format like JSON or XML** so it can be:

- Sent over the network
 - Saved to a file or database
-

What Is Deserialization?

Deserialization is the **reverse process**: converting **JSON/XML into a Java object** so the server can process it.

In Spring Boot:

Spring Boot (via **Jackson** by default) automatically:

- **Serializes** Java objects into JSON in HTTP **responses**
 - **Deserializes** JSON into Java objects in HTTP **requests**
-

Example: Student API

Step 1: Model Class

```
package com.example.demo.model;
```

```
public class Student {  
    private Long id;  
    private String name;  
    private String department;  
  
    // Constructors, Getters, and Setters  
    public Student() {}  
  
    public Student(Long id, String name, String department) {  
        this.id = id;  
        this.name = name;  
        this.department = department;  
    }  
  
    public Long getId() { return id; }  
    public void setId(Long id) { this.id = id; }  
  
    public String getName() { return name; }  
    public void setName(String name) { this.name = name; }  
  
    public String getDepartment() { return department; }  
    public void setDepartment(String department) { this.department = department; }  
}
```

Step 2: REST Controller

```
package com.example.demo.controller;

import com.example.demo.model.Student;
import org.springframework.web.bind.annotation.*;

@RestController
@RequestMapping("/api/students")
public class StudentController {

    // Deserialization: Accept JSON, convert to Java object
    @PostMapping
    public String createStudent(@RequestBody Student student) {
        // student object is now deserialized from JSON
        return "Received student: " + student.getName();
    }

    // Serialization: Return Java object, convert to JSON
    @GetMapping("/{id}")
    public Student getStudent(@PathVariable Long id) {
        return new Student(id, "Anjali", "CSE"); // automatically serialized to JSON
    }
}
```

Step 3: Test with Postman or curl

POST Request (Deserialization)

POST /api/students

Content-Type: application/json

```
{  
  "id": 101,  
  "name": "Anjali",  
  "department": "CSE"  
}
```

GET Response (Serialization)

GET /api/students/101

```
{  
  "id": 101,  
  "name": "Anjali",  
  "department": "CSE"  
}
```

Summary Table

Process	Direction	In Spring Boot
Serialization	Java → JSON	Response body (return from controller)
Deserialization	JSON → Java	Request body (e.g., @RequestBody)

Spring Boot handles this **automatically using Jackson**. If needed, you can customize JSON with annotations like:

- @JsonProperty
- @JsonIgnore
- @JsonFormat

8. JSON properties

When working with **JSON data in Spring Boot**, you often need to customize how Java fields are **serialized to JSON** and **deserialized from JSON**. Spring Boot uses **Jackson** by default for this purpose.

Common Jackson Annotations for JSON Properties

Annotation	Purpose
@JsonProperty	Rename the JSON field
@JsonIgnore	Exclude a field from JSON
@JsonInclude	Include only non-null or non-empty fields
@JsonFormat	Define custom date/time formats
@JsonIgnoreProperties	Ignore specific fields from incoming JSON

Example: Student Class with JSON Annotations

Student.java

```
package com.example.demo.model;

import com.fasterxml.jackson.annotation.*;
import java.time.LocalDate;

public class Student {

    @JsonProperty("student_id") // Rename field in JSON
    private Long id;

    private String name;

    @JsonIgnore // Don't include in JSON output
    private String password;
```

```

@JsonInclude(JsonInclude.Include.NON_NULL) // Only include if not null
private String department;

@JsonFormat(pattern = "dd-MM-yyyy") // Custom date format
private LocalDate dateOfBirth;

// Constructors, Getters, and Setters
public Student() {}

public Student(Long id, String name, String password, String department, LocalDate dateOfBirth)
{
    this.id = id;
    this.name = name;
    this.password = password;
    this.department = department;
    this.dateOfBirth = dateOfBirth;
}

// Getters and setters...
}

```

Sample JSON Output

If you return a Student object from a controller, it will look like this:

```

{
  "student_id": 101,
  "name": "Anjali",
  "department": "CSE",
  "dateOfBirth": "19-07-2002"
}

```

(Note: password is not included because of @JsonIgnore, and field names are mapped as specified.)

Sample REST Controller

```
@RestController
@RequestMapping("/api/students")
public class StudentController {

    @GetMapping("/{id}")
    public Student getStudent(@PathVariable Long id) {
        return new Student(
            id,
            "Anjali",
            "secret123",
            "CSE",
            LocalDate.of(2002, 7, 19)
        );
    }
}
```

Summary Table

Annotation	Effect
@JsonProperty("json_name")	Rename Java field in JSON
@JsonIgnore	Exclude from both serialization and deserialization
@JsonInclude	Skip null/empty fields
@JsonFormat	Format dates or numbers
@JsonIgnoreProperties({"field1", "field2"})	Ignore multiple fields from input

9. Managing data access.

In a Spring Boot application, data access is handled using the Spring Data JPA module (or JDBC, MongoDB, etc.), which abstracts database interaction with a clean, layered architecture.

Key Layers in Data Access Architecture

Layer	Purpose	Common Tool
Controller	Handles HTTP requests	@RestController
Service	Contains business logic	@Service
Repository	Communicates with the database (CRUD)	@Repository, JPA
Entity	Java class mapped to a database table	@Entity

Folder Structure Example

com.example.demo

```
|
|— controller/
|   └── StudentController.java
|— service/
|   └── StudentService.java
|— repository/
|   └── StudentRepository.java
|— model/
|   └── Student.java
└── DemoApplication.java
```

Example: Managing Student Data

Student Entity – Model Class

```
package com.example.demo.model;
import jakarta.persistence.*;

@Entity
public class Student {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String name;
    private String department;

    // Getters and Setters
}
```

StudentRepository – Data Access Layer

```
package com.example.demo.repository;

import com.example.demo.model.Student;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.stereotype.Repository;
```

@Repository

```
public interface StudentRepository extends JpaRepository<Student, Long> {
    // Custom query methods (if needed) can go here
}
```

StudentService – Business Logic Layer

```
package com.example.demo.service;

import com.example.demo.model.Student;
import com.example.demo.repository.StudentRepository;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Service;
import java.util.List;
```

@Service

```
public class StudentService {
```

@Autowired

```
    private StudentRepository studentRepository;
```

```
    public List<Student> getAllStudents() {
        return studentRepository.findAll();
    }
```

```

public Student getStudent(Long id) {
    return studentRepository.findById(id)
        .orElseThrow(() -> new RuntimeException("Student not found"));
}

public Student saveStudent(Student student) {
    return studentRepository.save(student);
}

public void deleteStudent(Long id) {
    studentRepository.deleteById(id);
}
}

```

StudentController – REST API Layer

```

package com.example.demo.controller;

import com.example.demo.model.Student;
import com.example.demo.service.StudentService;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.web.bind.annotation.*;
import java.util.List;

@RestController
@RequestMapping("/api/students")
public class StudentController {

    @Autowired
    private StudentService studentService;

```

```
@GetMapping
public List<Student> getAll() {
    return studentService.getAllStudents();
}
```

```
@GetMapping("/{id}")
public Student get(@PathVariable Long id) {
    return studentService.getStudent(id);
}
```

```
@PostMapping
public Student add(@RequestBody Student student) {
    return studentService.saveStudent(student);
}
```

```
@DeleteMapping("/{id}")
public void delete(@PathVariable Long id) {
    studentService.deleteStudent(id);
}
}
```

Spring Boot Auto Configuration for Database

In `application.properties`:

`properties`

`spring.datasource.url=jdbc:mysql://localhost:3306/school`

`spring.datasource.username=root`

`spring.datasource.password=admin`

`spring.jpa.hibernate.ddl-auto=update`

`spring.jpa.show-sql=true`

You can replace MySQL with H2, PostgreSQL, or any JDBC-supported DB.

Summary

Component	Role
@Entity	Maps Java object to DB table
JpaRepository	Handles CRUD without boilerplate
@Service	Applies business logic
@RestController	Exposes data through REST endpoints